

AI / AI ENGINEERING / LARGE LANGUAGE MODELS

How To Add Tool Support to AI Agents for Performing Actions

Tips on how to design and implement a robust tool system for AI agents, using examples that you can adapt for your own enterprise apps.

Dec 17th, 2024 9:00am by [Janakiram MSV](#)



Image via Unsplash+.

More from this series on AI agent development (download all the code from [GitHub](#)):

- Overview: [AI Agents: A Comprehensive Introduction for Developers](#)
- Step 1: [How To Define an AI Agent Persona by Tweaking LLM Prompts](#)
- Step 2: [Enhancing AI Agents: Adding Instructions, Tasks and Memory](#)
- Step 3: [Enhancing AI Agents: Implementing Reasoning Through Prompt Engineering](#)
- Step 4: [How To Add Persistence and Long-Term Memory to AI Agents](#)
- Step 5: [How To Add RAG to AI Agents for Contextual Understanding](#)
- Step 6: [How To Add Tool Support to AI Agents for Performing Actions](#)

In the process of developing a framework for implementing AI agents, we have explored core components like personas, instructions, tasks, and execution strategies. These elements shape the cognitive processes of the agents. However, in the modern interconnected enterprise environment, cognitive processes alone are insufficient. Agents must be able to act, extending beyond their internal knowledge to interact with the external world.

This is where tools come into play — they are the hands and eyes of our AI agents, extending their capabilities far beyond simple text generation and static knowledge cutoffs. Just as human employees rely on various software tools, databases, and APIs to accomplish their tasks, AI agents need similar capabilities to be truly effective in an enterprise setting.

- **Provide Current Information:** Instead of relying on training data that may be months or years old, agents can fetch the latest information in real-time.
- **Perform Complex Tasks:** By combining multiple tools, agents can handle sophisticated workflows that require interaction with various systems and services.
- **Validate and Verify:** Tools allow agents to fact-check their responses against authoritative sources, significantly improving accuracy and reliability.
- **Integrate with Enterprise Systems:** Agents can seamlessly work with existing enterprise infrastructure, from CRM systems to custom internal tools.
- **Scale Operations:** By automating interactions with various tools and services, agents can handle increased workloads without linear resource scaling.

However, implementing tool support isn't just about connecting APIs — it requires careful consideration of architecture, security, error handling, and user experience. In this article, we'll explore how to design and implement a robust tool system for AI agents, using practical examples that you can adapt for your own enterprise applications.

Understanding Tool Architecture

At its core, tool support consists of three main components:

- A base *Tool* class that defines the interface;
- Concrete tool implementations; and
- A registry system to manage available tools.

Let's break down each component and see how they work together.

The Base Tool Class

First, let's look at the abstract base class that all tools must implement:

```
from abc import ABC, abstractmethod
from typing import Any, Dict, Optional
from dataclasses import dataclass

@dataclass
class ToolResult:
    """Represents the result of a tool execution."""
    success: bool
    data: Any
    error: Optional[str] = None

class Tool(ABC):
    """Base class for all tools."""

    @property
    @abstractmethod
    def name(self) -> str:
        """The name of the tool."""
        pass

    @property
    @abstractmethod
    def description(self) -> str:
        """Description of what the tool does."""
```

```

@abstractmethod
def parameters(self) -> Dict[str, str]:
    """Dictionary of parameter names and their descriptions."""
    pass

@abstractmethod
def execute(self, **kwargs) -> ToolResult:
    """Execute the tool with the given parameters."""
    pass

def to_prompt_format(self) -> str:
    """Convert tool information to a format suitable for prompts."""
    params_str = "\n".join(f"  {name}: {desc}" for name, desc in self.parameters.items())
    return f"""Tool: {self.name}
Description: {self.description}
Parameters:
{params_str}"""

```

Tool Registry System

The tool registry manages the available tools and provides methods to access them:

```

class ToolRegistry:
    """Registry for managing available tools."""

    def __init__(self):
        self._tools: Dict[str, Tool] = {}

    def register(self, tool: Tool) -> None:
        """Register a new tool."""
        if not isinstance(tool, Tool):
            raise TypeError("Tool must be an instance of Tool class")
        self._tools[tool.name] = tool

    def get_tool(self, name: str) -> Optional[Tool]:
        """Get a tool by name."""
        return self._tools.get(name)

    def list_tools(self) -> List[str]:
        """List all registered tool names."""
        return list(self._tools.keys())

    def get_tools_prompt(self) -> str:
        """Get a formatted string of all tools for use in prompts."""
        if not self._tools:
            return "No tools available."

        tools_str = "\n\n".join(tool.to_prompt_format() for tool in self._tools.values())
        return f"""Available Tools:

{tools_str}

```

To use a tool, specify it in your response as:

```

Tool: [tool_name]
Parameters:
  - param1: value1
  - param2: value2
"""

```

Implementing Specific Tools

Let's now look at two concrete tool implementations: Wikipedia search and web search.

```
from typing import Dict, Any
from tools import Tool, ToolResult

class WikipediaTool(Tool):
    """Tool for searching Wikipedia"""

    @property
    def name(self) -> str:
        return "wikipedia_search"

    @property
    def description(self) -> str:
        return "Search Wikipedia for information about a topic"

    @property
    def parameters(self) -> Dict[str, Dict[str, Any]]:
        return {
            "query": {
                "type": "string",
                "description": "The Wikipedia search query"
            }
        }

    def execute(self, **kwargs) -> ToolResult:
        try:
            query = kwargs.get("query")
            print(f"Searching Wikipedia for: {query}")
            search_results = wikipedia.search(query)
            if not search_results:
                return ToolResult(
                    success=True,
                    data="No Wikipedia articles found for the query."
                )

            page = wikipedia.page(search_results[0])
            summary = page.summary[:500] + "..."

            return ToolResult(
                success=True,
                data=f"Title: {page.title}\nSummary: {summary}"
            )
        except Exception as e:
            return ToolResult(
                success=False,
                data="",
                error=f"Wikipedia search failed: {str(e)}"
            )
```

Web Search Tool

```
import os
from typing import Dict, Any
from tools import Tool, ToolResult
from tavily import TavilyClient

class WebSearchTool(Tool):
    """Tool for performing web searches using Tavily API"""

    def __init__(self):
        """Initialize the web search tool with API key."""
        self.api_key = os.getenv('TAVILY_API_KEY', '')
        if not self.api_key:
```

```
def name(self) -> str:
    return "web_search"

@property
def description(self) -> str:
    return "Search the web for information about a topic"

@property
def parameters(self) -> Dict[str, Dict[str, Any]]:
    return {
        "query": {
            "type": "string",
            "description": "The search query to look up"
        }
    }

def execute(self, **kwargs) -> ToolResult:
    try:
        query = kwargs.get("query")
        if not query:
            return ToolResult(
                success=False,
                data="",
                error="No query provided"
            )

        client = TavilyClient(api_key=self.api_key)
        search_response = client.search(query=query)

        # Take the top 3 results
        results = search_response['results'][:3]

        # Format results
        formatted_results = []
        for result in results:
            formatted_results.append({
                "title": result.get('title', 'No title'),
                "content": result.get('content', 'No content'),
                "url": result.get('url', 'No URL')
            })

        formatted_output = self._format_search_results(formatted_results)

        return ToolResult(
            success=True,
            data=formatted_output
        )
    except Exception as e:
        return ToolResult(
            success=False,
            data="",
            error=f"Web search failed: {str(e)}"
        )
```

Using Tools With Agents

Here's how to integrate tools with an agent:

```
from agent import Agent
from wikipedia_tool import WikipediaTool
from websearch_tool import WebSearchTool
```

```
wiki_tool = WikipediaTool()
web_tool = WebSearchTool()

# Register tools with the agent
agent.tools = [wiki_tool, web_tool]

# Set up agent persona
agent.persona = """I am a research assistant with access to both Wikipedia and web search.
I can find information from multiple sources to provide comprehensive answers."""

# Execute a task
response = agent.execute("What are the latest developments in quantum computing?")
print(response)
```

How Tool Execution Works

When an agent uses a tool, the process follows these steps:

- The agent receives a task and determines if it needs to use a tool.
- If a tool is needed, the agent formats its request using the specified format:
CopyTool: [tool_name]
Parameters:
- param1: value1
- The agent's response is parsed to extract tool usage information.
- The tool is executed with the provided parameters.
- The tool's result is incorporated into the agent's response.

The `parse_tool_usage` function handles extracting tool information from the agent's response:

TRENDING STORIES

1. [How to kill the code review](#)
2. ["Code should be regenerated, not maintained": Codeplain makes the case for spec-driven development](#)
3. [Coinbase runs 1,200 agents and just slashed its AI bill in half](#)
4. [How long before we stop reading the code?](#)
5. [Your engineering org needs an AI slop registry](#)

```
def parse_tool_usage(response: str) -> Optional[Dict[str, Any]]:
    """Parse a response string to extract tool usage information."""
    try:
        if "Tool:" not in response:
            return None

        lines = response.split('\n')
        tool_info = {}

        # Find tool name
        for i, line in enumerate(lines):
            if line.startswith("Tool:"):
                tool_info["name"] = line.replace("Tool:", "").strip()
                break
```

```
for line in lines:
    if ":" in line and "-" in line:
        param_line = line.split(":", 1)
        param_name = param_line[0].replace("-", "").strip()
        param_value = param_line[1].strip()
        params[param_name] = param_value

tool_info["parameters"] = params
return tool_info
except Exception:
    return None
```

Best Practices for Tool Implementation

- **Error Handling:** Always wrap tool execution in try-except blocks and return meaningful error messages.
- **Clear Documentation:** Provide clear descriptions and parameter specifications for each tool.
- **Consistent Interface:** Follow the Tool base class interface consistently.
- **Result Formatting:** Format tool results in a clear, readable way.
- **Resource Management:** Handle API keys and external resources securely.
- **Modularity:** Keep tool implementations independent and focused on a single responsibility.

Conclusion

Tool support is a crucial feature that makes AI agents more capable and practical for real-world applications. By following the patterns and practices outlined in this article, you can create a robust and extensible tool system for your agents. The combination of a clear base interface, an efficient registry system, and well-implemented concrete tools provides a solid foundation for building sophisticated agent capabilities.

Remember that tools should be designed to be:

- Reusable across different agents;
- Easy to maintain and update;
- Well-documented for other developers; and
- Robust in handling errors and edge cases.

With these principles in mind, you can create powerful tools that enhance your agents' capabilities and make them more useful for real-world tasks.

This concludes the series on AI agents, where we implemented an end-to-end framework to explore crucial capabilities and functions agents.

TNS



Janakiram MSV (Jani) is a practicing architect, research analyst, and advisor to Silicon Valley startups. He focuses on the convergence of modern infrastructure powered by cloud-native technology and machine intelligence driven by generative AI. Before becoming an entrepreneur, he spent...

[Read more from Janakiram MSV →](#)

TNS owner Insight Partners is an investor in: Class, Tavily.

TRENDING STORIES

1. How to kill the code review
2. "Code should be regenerated, not maintained": Codeplain makes the case for spec-driven development
3. Coinbase runs 1,200 agents and just slashed its AI bill in half
4. How long before we stop reading the code?
5. Your engineering org needs an AI slop registry

INSIGHTS FROM OUR SPONSORS



Comparing Kubernetes Development Tools in 2026: Telepresence, mirrord, Okteto, and Signadot

18 June 2026

Watch Claude Code Validate a Microservices Change in a Closed Loop

4 June 2026

Signadot Plans

20 May 2026



Introducing SonarQube Hunter Agent(beta): Catch logic flaws, as you code.

9 July 2026

More than just data: The hidden security risks in Jupyter notebooks

6 July 2026

SonarQube plugins bring trusted verification to Claude Code, Copilot, Codex, Cursor, and beyond

1 July 2026

[Advancing humanitarian protection online](#)

9 July 2026

[Responsibly building the AI future](#)

9 July 2026

[Bringing Ode Poetry to life with MAI's audio models](#)

9 July 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the most recent TNS articles in your inbox each day.

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks

Kubernetes
Observability
Operations
Platform Engineering

newsletter
TNS RSS Feeds

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

 roadmap.sh

Community created roadmaps, articles, resources
and journeys for developers to help you choose your
path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS



© The New Stack 2026

Disclosures
Terms of Use
Advertising Terms & Conditions
Privacy Policy
Cookie Policy